

Starting and ending blocks: You just saw a block used with the `.times` method on an integer. The block starts with the keyword 'do' and ends with the keyword 'end'. The *do/end* style is always acceptable.

Bracket blocks: When a block contains just a single instruction, though, we often use the alternate markers '{' and '}' to begin and end the block:

```
5.times{ puts "Hello, World!" }
```

6. Arrays

Usually, when we're writing a program it's because we need to deal with a *collection* of data. There are lots of cool things to do with an array. Here are a few examples.

.sort: The `sort` method will return a new array where the elements are sorted. If the elements are strings, they'll come back in alphabetical order. If they're numbers, they'll come back in ascending value order. Try these:

```
C:\Users\Administrator>irb
irb(main):001:0> one = ["this", "is", "an", "array"]
irb(main):002:0> one.sort
irb(main):003:0> one
```

You can rearrange the order of the elements using the `sort` method. You can iterate through each element using the `each` method. You can mash them together into one string using the `join` method. You can find the address of a specific element by using the `index` method. You can ask an array if an element is present with the `include` method. We use arrays whenever we need a list in which the elements are in a specific order.

7. Hashes

A hash is a collection of data, in which each element of the data is addressed by a *name*. A hash is an *unordered* collection, where the data is organised into 'key/value pairs'. Hashes have a more complicated syntax that takes some getting used to:

```
C:\Users\Administrator>irb
irb(main):001:0> produce = {"apples" => 3, "kiwi" => 1}
irb(main):002:0> puts "There are #{produce['apples']} apples in the fridge."
```

Simplified hash syntax: We commonly use symbols as the keys of a hash. When all the keys are symbols, then there is a shorthand syntax that can be used:

```
C:\Users\Administrator>irb
irb(main):001:0> produce = {"apples" => 3, "kiwi" => 1}
irb(main):002:0> puts "There are #{produce[:apples]} apples in the fridge."
```

Notice that the keys end with a colon rather than beginning with one, even though these are symbols.

8. Conditionals

Conditional statements evaluate to *true* or *false*. The most common conditional operators are `==` (equal), `>` (greater than), `>=` (greater than or equal to), `<` (less than), and `<=` (less than or equal to).

Conditional branching/instructions: Why do we have conditional statements? Most often, it's to control conditional instructions, especially *if/elsif/else* structures. Let's write an example by adding a method like this in IRB:

```
def water_status(minutes)
  if minutes < 7
    puts "The water is not boiling yet"
  elsif minutes == 7
    puts "It's just barely boiling"
  elsif minutes == 8
    puts "It's boiling!"
  else
    puts "Hot! Hot! Hot!"
  end
end
```

Try running the method with `water_status(5)`, `water_status(7)`, `water_status(8)` and `water_status(9)`.

Understanding the execution flow: When the minutes are 5, here is how the execution goes: "Is it true that 5 is less than 7? Yes, it is; so print the line *The water is not boiling yet*".

When the minutes are 7, it goes like this: "Is it true that 7 is less than 7? No. Next, is it true that 7 is equal to 7? Yes, it is; so print the line *It's just barely boiling*".

When the minutes are 8, it goes like this: "Is it true that 8 is less than 7? No. Next, is it true that 8 is equal to 7? No. Next, is it true that 8 is equal to 8? Yes, it is; so print the line *It's boiling!*".

Lastly, when the total is 9, the execution goes like this: "Is it true that 9 is less than 7? No. Next, is it true that 9 is equal to 7? No. Next, is it true that 9 is equal to 8? No. Since none of these are true, execute the *else* and print the line *Hot! Hot!*". 

By Miren Karamta

The author is a project scientist and IT systems manager at the Bhaskaracharya Institute for Space Applications and Geo-informatics (BISAG), Gandhinagar, Gujarat. You can reach him via email at mirenkaramta@yahoo.com. LinkedIn: <https://in.linkedin.com/in/miren-karamta-2b929122>